

Empiricist

18+3 · WEIGHT 3 LONGEST (Nykolos Rekasius)

Reproducible small-WM recipe · RC1-RC5 smoke honesty · hard numbers ·
stability + GAP · contrast vs DINO-WM / V-JEPA 2-AC

Course smoke $\neq$ paper 96%. Do NOT claim absolute Push-T SR from smoke. Cite Tab.5 / Ablation Bot for paper numbers. Kanta NOT seated.

SPINE: Stable E2E JEPA WM · MSE+SIGReg only · $\sim 15\text{M}$ · $48\times$ vs DINO-WM (0.98s/47s) · SIBLING to Meta JEPA · SIM-ONLY · PushT 96 ± 2.83 · fixed-FLOP 90vs13 · recon hurts · λ -only

Talk arc — Empiricist densest

Recipe → numbers → RC smokes → GAP → contrast → exam

02-04

Recipe + Fig.1

ViT-Tiny+AdaLN · MSE+ λ SIGReg · CEM 300×30 H=5 · no EMA/SG/freeze

05-07

Hard numbers + speed + SR

15M · 0.98s/47s · 90/13 · PushT 96±2.83 · Fig.3/6

08-12

RC1-RC5 smoke cards

dropout · decoder · CEM vs Adam · λ · speed proxy

13-16

GAP + contrast + CEM Fig.4

no EMA/SG factorial · TwoRoom · LeWM|DINO|2-AC

17-18

Tattoo + close

exam: reproducible small-WM; sim E2E $\neq$ 2-AC

Reproducible recipe card

Implement this — Ablation Bot / Empiricist pack

Data Offline (o,a) only · reward-free · frame-skip 5 · 224^2 · bs 128 · $T_{\text{sub}}=4$ · 10 ep typical

Encoder ViT-Tiny /14 · 12L · 3H · dim 192 ($\sim 5\text{M}$) · [CLS] $\rightarrow$ MLP+BatchNorm projector

Predictor Transformer 6L · 16H · dropout 0.1 ($\sim 10\text{M}$) · AdaLN actions (zero-init) · history $N=3/1$

Loss $L = \text{MSE}(\hat{z}_{t+1}, z_{t+1}) + \lambda \text{SIGReg}(Z)$ · $M=1024$ · $\lambda=0.1$ default · λ -only knob

Do NOT EMA target · stop-grad · frozen DINOv2 · VICReg 7-term · pixel recon (unless ablating)

Plan CEM 300 samples · 30 iters PushT / 10 else · elite 30 · $H=5$ · cost $\|\hat{z}_H - z_g\|_2^2$ · MPC

Code github.com/lucas-maes/le-wm · le-wm.github.io · Hydra config/train/

Fig. 1 — Training pipeline

Encoder $\rightarrow z$ · Predictor(z, a) $\rightarrow \hat{z}$ · MSE + SIGReg projections · no EMA/SG

arXiv:2003.19312v3 [cs.LG]

end-to-end from raw pixels using only two loss terms: a next embedding prediction loss and a regularizer enforcing Gaussian-distributed latent embeddings. This reduces tunable loss hyperparameters from six to one compared to the only existing end-to-end alternative. With 15M parameters trainable on a single GPU in a few hours, LeWM plans up to $48\times$ faster than foundation-model-based world models while remaining competitive across diverse 2D and 3D control tasks. Beyond control, we show that LeWM's latent space encodes meaningful physical structure through probing of physical quantities. Surprise evaluation confirms that the model reliably detects physically implausible events. Code available [here](#).

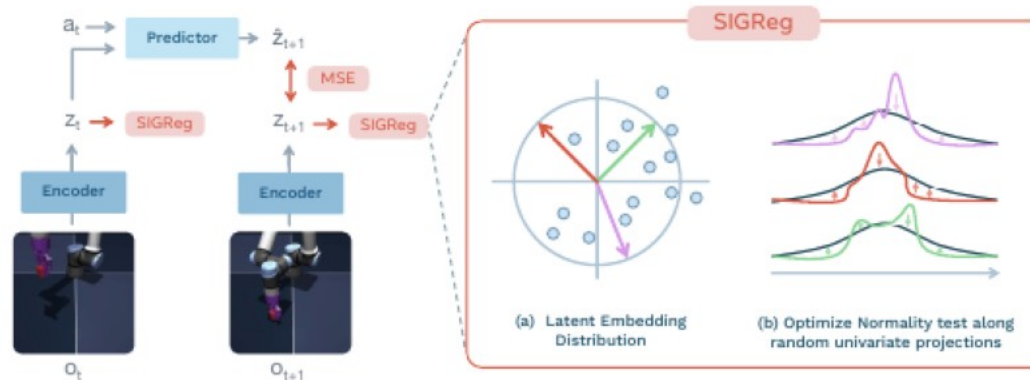


Figure 1: LeWorldModel Training Pipeline. Given frame observations $\mathbf{o}_{1:T}$ and actions $\mathbf{a}_{1:T}$, the encoder maps frames into low-dimensional latent representations $\mathbf{z}_{1:T}$. The predictor models the environment dynamics by autoregressively predicting the next latent state $\mathbf{z}_{t+1}$ from the current latent state $\mathbf{z}_t$ and action $\mathbf{a}_t$. The encoder and predictor are jointly optimized using a mean-squared error (MSE) prediction loss. LeWM does not rely on any training heuristics, such as stop-gradient, exponential moving averages, or pre-trained representations. To prevent trivial collapse, the SIGReg regularization term enforces Gaussian-distributed latent embeddings, promoting feature diversity. For tractability, latent embeddings are projected onto multiple random directions, and a normality test is applied to each one-dimensional projection. Aggregating these statistics encourages the full embedding distribution to match an isotropic Gaussian.

* Equal contribution. Correspondence to lucas.maes@mila.quebec

Paper Fig.1 — end-to-end from pixels; SIGReg = random projections + Epps-Pulley normality.

Hard numbers table

Quote these; source in right column

Claim	Number	Source
Params	~15M	Abstract / §1
Plan latency	0.98 s vs DINO 47 s (~48×)	Fig. 3
Tokens	~200× fewer than DINO-WM	Fig. 3 caption
Fixed-FLOP Push-T	LeWM 90 vs DINO 13	Fig. 3
Fixed-FLOP Cube	LeWM 74 vs DINO 48	Fig. 3
Push-T SR 3-seed	96.0±2.83 PLDM 78±5 DINO 92±1.63	Tab. 5
λ robust band	SR>80% for λ∈[0.01,0.2]; break @0.5	Fig. 16
Recon ablation	96.0 → 86.0 (±7.54) with decoder loss	Tab. 7
Dropout	0→78; 0.1→96; 0.2→85; 0.5→67	Tab. 9
Encoder / pred size	ViT 96 / RN18 94 · tiny 80.7 / small 96 / base 86.7	Tab. 8 / 6
CEM vs Adam	96 vs 84 (LeWM); SGD 26	Tab. 10
HP complexity	LeWM O(log n) vs PLDM O(n ⁶)	§4.3

Fig. 3 — Planning speed & fixed-FLOP

0.98 s vs 47 s · $\sim 48\times$ · Push-T 90 vs 13 · Cube 74 vs 48 · $\sim 200\times$ fewer tokens



Figure 3: **Planning time and performance under fixed compute.** **Left:** Planning time comparison averaged over 50 runs. Encoding observations with $\sim 200\times$ fewer tokens than DINO-WM allows LeWM to achieve planning speeds comparable to PLDM while being up to $\sim 50\times$ faster than DINO-WM. **Center-Right:** Planning performance under the same computational budget (fixed FLOPs). LeWM significantly outperforms DINO-WM on Push-T (center) and OGBench-Cube (right). See App. D for planning setup details.

- We evaluate physical understanding in the latent space through probing of physical quantities and a violation-of-expectation test for detecting unphysical trajectories.

2 Related Work

World Models aim to learn predictive models of environment dynamics from data, enabling agents to reason about future states in imagination. A prominent class of WMs consists of *generative* approaches that explicitly model environment dynamics in pixel space. These action-conditioned generative models act as learned simulators by producing future observations conditioned on past

Speedup = cheaper compact z dynamics steps, not fewer CEM iters. RC5 = timing proxy only.

Fig. 6 — Planning SR across envs

Push-T 96 · Reacher ~86 · Cube ~74 (DINO ahead) · TwoRoom ~87 (behind DINO/PLDM)

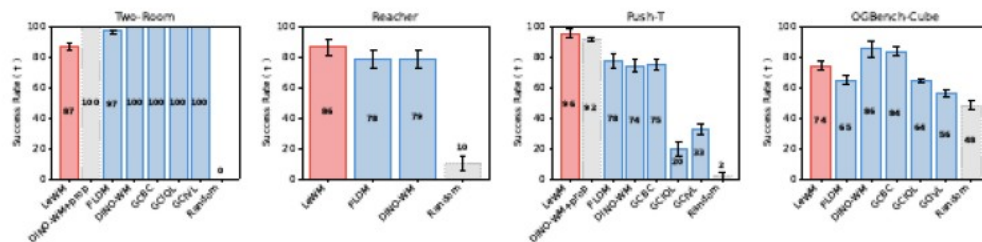


Figure 6: **Planning performance across environments.** Results are shown for *Two-Room* (left), *Reacher* (center 1), *PushT* (center-2) and *OGBench-Cube* (right). LeWM consistently outperforms PLDM and DINO-WM on Push-T and Reacher. On OGBench-Cube, DINO-WM slightly outperforms LeWM, possibly due to the higher visual complexity and the 3D nature of the environment, which makes encoder training more challenging. In the simpler Two-Room environment, PLDM and DINO-WM outperform LeWM, which may be explained by the SIGReg regularization encouraging a Gaussian distribution in a high-dimensional latent space, while the intrinsic dimensionality of the environment is much lower.

DINO-WM. Notably, on PushT, LeWM (pixels-only) surpasses DINO-WM even when DINO-WM has access to additional proprioceptive information, demonstrating LeWM’s ability to capture underlying task-relevant quantities. Interestingly, LeWM performs worse on the simplest environment, Two-Room. A possible explanation is that the low diversity and low intrinsic dimensionality of this dataset make it difficult for the encoder to match the isotropic Gaussian prior enforced by SIGReg in a high-dimensional latent space, which may lead to a less structured latent representation. This highlights a potential limitation of the SIGReg regularization in very low-complexity environments.

Moreover, when comparing planning speedups (Fig. 3), LeWM achieves a $48\times$ faster planning time, with the full planning completing in under one second while preserving competitive performance across tasks. This planning time remains consistent across environments for a fixed planning setup, narrowing the gap toward real-time control.

4.3 Towards Stable Training of World Models

Ablations. We perform ablations on several design choices of LeWM. First, we analyze the sensitivity of SIGReg to its internal parameters, namely the number of random projections and the number of integration knots. The performance is largely unaffected by these quantities, indicating that they do not require careful tuning. As a result, the regularization weight λ remains the only effective hyperparameter. Since only a single hyperparameter needs to be tuned, grid search can be performed efficiently using a simple bisection strategy ($\mathcal{O}(\log n)$), whereas PLDM requires search in polynomial time ($\mathcal{O}(n^6)$). We also study the effect of the embedding dimensionality. While the representation dimension must be sufficiently large for the method to perform well, performance

Honesty: TwoRoom underperforms; Cube trails DINO slightly. Push-T +18 pp vs PLDM.

RC1 — dropout sweep · Ablation Bot #1 PRIMARY

Tab. 9 · course smoke · directional ranking only

p=0.0

78±6.54

Push-T SR (paper)

p=0.1 ★

96.0±2.83

Push-T SR (paper)

p=0.2

85.33±5.74

Push-T SR (paper)

p=0.5

66.67±4.11

Push-T SR (paper)

Smoke pass

ViT-Tiny+pred · $\lambda=0.1$ · no decoder · $p \in \{0, 0.1, 0.2\}$ · 1 seed · short steps. Pass: 3 runs finish; $p=0.1$ best among trio; plot SR/cost vs p .

Honesty: NOT required to hit 96% absolute. Ranking moral: $0.1 \geq 0.2 \geq 0.0 \gg 0.5$. Budget ~3-10 GPU-h.

RC2 — decoder_loss_ablation · Ablation Bot #2 PRIMARY

Tab. 7 · JEPA vs generative · recon hurts planning

w/o decoder loss

96.0 ± 2.83

default JEPA (paper)

with decoder loss

86.0 ± 7.54

higher variance too

Smoke: toggle recon_loss on/off only ($p=0.1$, $\lambda=0.1$). Pass: w/o $\geq$ with directional. Decoder must be train-time loss to match Tab.7 — post-hoc viz $\neq$ ablation.

Budget $\sim 2-8$ GPU-h. Cross-link: I-JEPA emb $\gg$ pixels; V-JEPA dense $\neq$ pixel recon.

RC3 — planning_solver CEM vs Adam · Ablation Bot #4

Tab. 10 · fixed ckpt · eval-only

CEM (default)

96.0 ± 2.83

300 pop · 30 iters · elite 30 · H=5

Adam

84 ± 7.12

match wall-time or #model-calls

RMSProp

67.33 ± 2.49

optional extended grid

SGD

26 ± 4.32

worst — document if swept

Smoke: best RC1/RC2-off ckpt · no retrain · CEM > Adam gap · ≤1-4 GPU-h. Do not claim PLDM Tab.10 cells without PLDM.

RC4 — lambda sweep · Ablation Bot #5 PRIMARY

Fig. 16 · only effective loss HP → bisection $O(\log n)$

$\lambda=0.01$

healthy (>80%)

$\lambda \approx 0.09$

peak band

$\lambda=0.1$

default

$\lambda=0.5$

cliff / degrade

Smoke grid {0.01, 0.09, 0.5}; optional $M \in \{64, 1024\}$ flat.
Pass: 0.5 clearly worse. ~3-10 GPU-h.

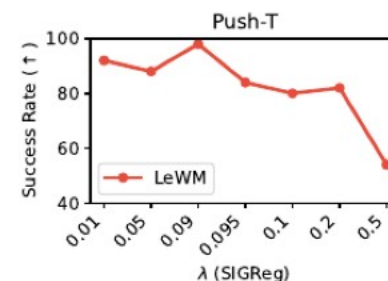


Figure 16: Effect of the SIGReg regularization weight λ on Push-T planning performance. Success rate remains above 80% across a wide range of values ($\lambda \in [0.01, 0.2]$), peaking near $\lambda = 0.09$. Performance degrades sharply only at $\lambda = 0.5$, where the regularizer dominates the prediction loss and hinders dynamics modeling. Since λ is the only effective hyperparameter of LeWM, the SIGReg loss coefficient is easy to tune via a simple bisection search.

Table 7: Effect of adding a reconstruction loss during training. We report the success rate (SR) on the Push-T planning task. The model trained without the decoder loss achieves higher performance.

	Push-T (SR ↑)
LeWM w/o decoder loss	96.0 ± 2.83
LeWM with decoder loss	86.0 ± 7.54

RC5 — planning_speed_proxy · Ablation Bot #9 Partial

Fig. 3 morals · NOT full DINO-WM bake-off

47 s

DINO-WM full plan

0.98 s

LeWM full plan

~48×

speedup claimed

~200×

fewer tokens

90 vs 13

fixed-FLOP Push-T

74 vs 48

fixed-FLOP Cube

Smoke: timing + token-count vs high-token frozen-encoder stub. Pass: compact $\ll$ stub wall-time. $\leq 1-4$ GPU-h.

Caveat: speedup = cheaper z, not fewer CEM iters.

Heuristic GAP + stability narrative

Ablation Bot B9 — do not invent EMA/SG factorial as paper ablate

GAP — extension only

Paper claims no stop-grad / EMA / pretrained encoder but does NOT report a within-LeWM factorial adding them back. Any such run = extension, not Ablation Bot rank.

Supportive evidence (indirect)

Tab.5 low variance · Fig.18–19 smooth 2-term vs PLDM noisy · λ plateau · M/knots inert · ResNet works.

Caveats + allowed stabilizers

TwoRoom underperforms; Cube favors DINO; speedup=cheaper z. “No heuristics” still uses dropout 0.1 + AdaLN zero-init + BN projector — document as allowed stabilizers (KANTA_PACK).

RC naming: follow DELIVERABLE RC1–RC5 (dropout/decoder/CEM/ λ /speed) — Empiricist Nykolas weight 3.

Contrast — LeWM vs DINO-WM vs V-JEPA 2-AC

Precision for club · do not collapse recipes

Axis	LeWM	DINO-WM	V-JEPA 2-AC
Encoder	Joint E2E	Frozen DINOv2	Video SSL then AC
Collapse	SIGReg	Freeze	EMA/SG in SSL
Latent	Compact CLS ~192d	Patch tokens ~200×	Video / AC tokens
Robot	SIM ONLY	Sim (design ZS)	CLEAR Franka
Plan speed	0.98 s	47 s	Separate (Franka CEM)
Niche	Tiny AC WM recipe	Foundation freeze	Real-robot JEPA plan

Fig. 4 — Latent CEM planning loop

Encode o_1, o_g · AR rollout · cost $\|\hat{z}_H - z_g\|^2$ · CEM 300×30 · $H=5$ · MPC

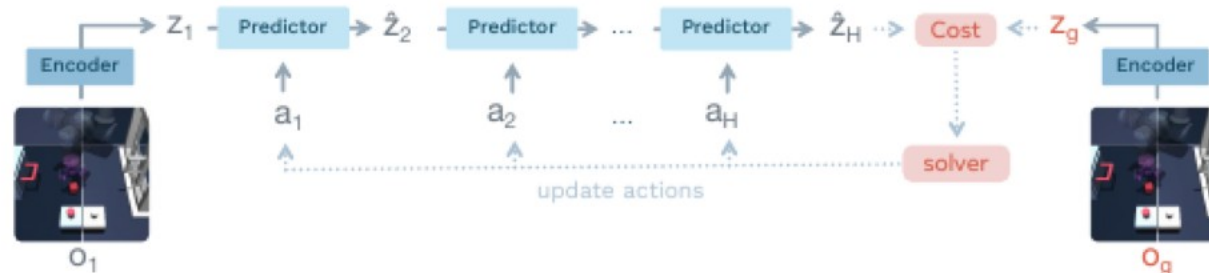


Figure 4: **Latent Planning with LeWorldModel**. Given an initial observation o_1 and a goal o_g , the world model learned in Fig. 2 performs planning in the LeWM latent space. The initial state embedding z_1 and the goal embedding z_g are obtained from the encoder. The predictor then rolls out future latent states up to a horizon H . A latent cost between the final predicted state and the goal embedding guides a solver to optimize the action sequence. This prediction–optimization loop is repeated until convergence to a good plan candidate.

instead perform planning directly in the latent space at test time using Model Predictive Control (MPC) [30–33, 18, 22].

3 Method: LeWorldModel

In this section, we introduce LeWorldModel (LeWM). We first describe the streamlined training procedure used to learn the latent world model from offline data, including the dataset, model architecture, and training objective. We then explain how the learned model can be leveraged for decision making through latent planning using model predictive control (MPC).

3.1 Learning the Latent World Model

Offline Dataset. We consider a fully offline and reward-free setting. LeWorldModel is trained

World-model params frozen at plan time. Speedup = cheaper compact z steps (not fewer CEM iters).

Ablations Fig.15 + training curves 18-19

Capacity / M / knots inert · LeWM smooth vs PLDM noisy

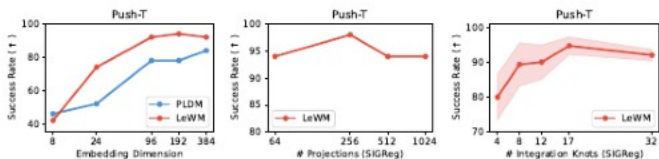


Figure 15: **Ablation studies of key design choices in LeWM.** **Left:** effect of the embedding dimension; performance improves with larger embeddings but quickly saturates beyond a certain threshold. **Center:** effect of the number of random projections used in SIGReg; performance remains stable, indicating that this parameter is not critical. **Right:** effect of the number of integration knots used to compute the SIGReg loss; results are similarly insensitive to this parameter.

Table 5: **Training Variance.** We report the mean success rate across three training seeds and the corresponding variance, evaluated over the same set of 50 trajectories on Push-T. The goal configuration is reachable within 25 steps, and we allow a planning budget of 50 steps. PLDM exhibits higher variance compared to DINO-WM and LeWM.

Model	Push-T (SR $\uparrow$)
DINO-WM	92.0 $\pm$ 1.63
PLDM	78.0 $\pm$ 5.0
LeWM (ours)	96.0 $\pm$ 2.83

Embedding dimensions. We study the impact of the embedding dimensionality on performance. As shown in Fig. 15, performance drops when the embedding dimension falls below a certain threshold (around 184), while increasing the dimension beyond this value yields diminishing returns and leads to performance saturation.

Number of projections in SIGReg. We study the impact of the number of projections used in

Figure 17: **Temporal Latent Straightening on Push-T.** Mean cosine similarity between consecutive latent velocity vectors (Eq. 9) over training. Higher values indicate straighter latent trajectories. PLDM explicitly encourages temporal regularity through a dedicated temporal smoothness loss ($\mathcal{L}_{\text{time-sm}}$), yet LeWM achieves substantially straighter latent paths as a purely emergent phenomenon, without any temporal regularization term in its objective.

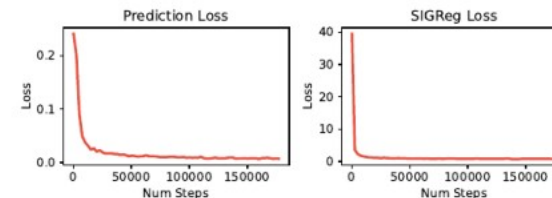


Figure 18: **Push-T Training curves for LeWM.**

Fig.15: emb dim drops below ~ 184 ; M & knots flat $\rightarrow \lambda$ -only. Allowed stabilizers (not “zero heuristics”): dropout 0.1, AdaLN zero-init, BN projector. Fig.18-19: smooth 2-term vs PLDM noisy.

Numbers tattoo — pin these

Shared spine across all five seats

~15M

params · 1 GPU / few h

0.98s vs 47s

~48× planning speed

~200×

fewer tokens vs DINO

90 vs 13

fixed-FLOP Push-T

74 vs 48

fixed-FLOP Cube

96.0±2.83

Push-T SR (Tab.5)

78±5 / 92±1.63

PLDM / DINO Push-T

$\lambda \in [0.01, 0.2]$

SR>80%; peak ~0.09

96 → 86

recon loss hurts (T7)

SIBLING

not Meta descendant

SIM-ONLY

cite 2-AC for robots

λ -only

vs PLDM 6 knobs

Empiricist — close

Takeaways + exam bite

1. Recipe: ViT-Tiny+AdaLN · MSE+ λ SIGReg · CEM 300×30 H=5 · no EMA/SG/freeze.
2. Must numbers: 15M · 0.98s/47s · 90/13 · PushT 96±2.83 · λ band · recon hurts · TwoRoom caveat.
3. RC1-RC5 (DELIVERABLE): dropout · decoder · CEM vs Adam · λ · speed — directional only.
4. GAP: no EMA/SG factorial (extension). Allowed stabilizers: dropout/AdaLN/BN. SIBLING + SIM-ONLY.

EXAM BITE

Reproducible small-WM recipe: two losses, one knob, sub-second CEM, Push-T 96% paper — but sim E2E JEPA-WM, not a Meta 2-AC replacement. Cite 2-AC for real-robot JEPA planning.